



Foldable iPhone

Apple analyst Ming-Chi Kuo says that Apple may release an 8-inch foldable iPhone in 2023. <http://dgit.in/may21-03>



EV for ride-hail drivers

Arrival, the electric vehicle manufacturer, is partnering with Uber to create an EV for ride-hail drivers. <http://dgit.in/may21-04>

```
const items = ['Milk', 'Bread', 'Cheese'];

function makeRisotto(item1, item2, item3) {
  console.log(
    'Please wait for a while as we are trying to process ${item1}, ${item2}, ${item3}'
  );
}

//Prior to ES6
makeRisotto(items[0], items[1], items[2]);
//ES6
makeRisotto(...items);
```

be to pass every item in the array as separate arguments to the function, separating them by commas. But ES6 allowed this particular operation to be very succinct by introducing the spread operator where we only need to write ... and the variable name. What happens behind the scenes is that all the elements within the items array are passed down to the function and are accepted separately as arguments and the execution proceeds normally. In hindsight, this way of handling parameters can be a life-saver on multiple real life instances where there might be several parameters that need passing into functions, as opposed to this example. Another use-case of spreading would be creating shallow copies and ultimately merging arrays entirely.

```
const items = ['Milk', 'Bread', 'Cheese'];
const locations = ['Delhi', 'Mumbai', 'kolkata'];
const newArray = [...items, ...locations];
const itemsCopy = [...items];
console.log(newArray);
console.log(itemsCopy);
```

```
> (6) ["Milk", "Bread", "Cheese", "Delhi", "Mumbai", "kolkata"]
> (3) ["Milk", "Bread", "Cheese"]
```

A quick glance to how we made the itemsCopy array shows how simple it is to just have a copy of the items array. We could have done the same using-

```
const itemsCopy = [items[0], items[1], items[2]];
```

However, the below code does not at all depend on us manually writing every element, yet allows us to create a copy entirely-

```
const items-
Copy = [...items];
```

Similarly, the newArray was created with the help of merging the items and the locations array. With EcmaScript 2018, spread operator was also enabled on objects, despite objects not being iterables.

```
const book = {
  name: 'Alice in Wonderland',
  author: 'Lewis Carrol',
}

const newBook = {...book, publishedIn: 1994}
console.log(newBook)
```

In the example, we have a book object and we create another object called newBook in which we want to make a copy of the book object and additionally have another property called publishedIn. This is how we do it. We spread the book object and as the default norms of creating an object go, place a new property with its value (for the publishedIn property), and voila! We have our newBook object ready. In

other words, JavaScript copied every property and its value, separated by commas from the book object and placed it in the newBook object while it was being created.

Let's understand

the REST operator now. The operator looks exactly the same as the spread operator with the only distinguishing factor being the position of its usage, which is on the left of the assignment operator. The example on the right is also a fine

demonstration of a de-structuring assignment combined with the rest operator.

If you notice on the left side of the de-structuring assignment, you'll find that there's a variable called otherBook, which we use to collect every other book there is except the firstBook and the secondBook. In essence, JavaScript, behind the scenes, packs every other element in the children-

Books array from the third element onwards and dumps them in the otherBooks array, which was something that we could do manually, separating each element by a comma. But rest takes away all the effort.

We can also combine rest and spread for it to look like the example below where we first combine every element in childrenBooks with classicBooks and then have the de-structuring assignment pick up the first, third and the remaining elements. The output for the same is quite intuitive and is shown as-

```
> (4) ["Duck Tales", "Talespin", "Wuthering heights", "Count of monte cristo"]
```

Another interesting use-case for the rest operator is seen while we deal with functions.

We see here that the function getSquare() is called four times with different numbers of parameter on

```
const childrenBooks = ['Heidi', 'Tarzan', 'Mowgli', 'Duck Tales', 'Talespin'];
const classicBooks = ['Wuthering heights', 'Count of monte cristo'];

const [firstBook, secondBook, ...otherBooks] = childrenBooks;
console.log(firstBook);
console.log(secondBook);
console.log(otherBooks);

const childrenBooks = ['Heidi', 'Tarzan', 'Mowgli', 'Duck Tales', 'Talespin'];
const classicBooks = ['Wuthering heights', 'Count of monte cristo'];

const [book1, , book3, ...others] = [...childrenBooks, ...classicBooks];
console.log(book1, book3);
console.log(others);
```